E-Commerce Customer Churn Analysis

DATA SET:

```
USE ecomm;

Describe customer_churn;
```

Field	Type	Null	Key	Default
CustomerID	int	NO	PRI	NULL
Churn	bit(1)	YES		NULL
Tenure	int	YES		NULL
PreferredLoginDevice	varchar(20)	YES		NULL
CityTier	int	YES		NULL
WarehouseToHome	int	YES		NULL
PreferredPaymentMode	varchar(20)	YES		NULL
Gender	enum('Male','Female')	YES		NULL
HourSpendOnApp	int	YES		NULL
NumberOfDeviceRegistered	int	YES		NULL
PreferedOrderCat	varchar(20)	YES		NULL

Data Cleaning:

Handling Missing Values and Outliers:

- Impute mean for the following columns, and round off to the nearest integer if required: WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear, DaySinceLastOrder.

```
UPDATE customer_churn c
JOIN (
    SELECT
        ROUND(AVG(WarehouseToHome)) AS Mean_WarehouseToHome,
        ROUND(AVG(HourSpendOnApp)) AS Mean_HourSpendOnApp,
        ROUND(AVG(OrderAmountHikeFromlastYear)) AS Mean_OrderAmountHike,
        ROUND(AVG(DaySinceLastOrder)) AS Mean_DaySinceLastOrder
    FROM customer_churn
) m
ON 1=1
SET c.WarehouseToHome = COALESCE(c.WarehouseToHome, m.Mean_WarehouseToHome),
    c.HourSpendOnApp = COALESCE(c.HourSpendOnApp, m.Mean_HourSpendOnApp),
    c.OrderAmountHikeFromlastYear = COALESCE(c.OrderAmountHikeFromlastYear, m.Mean_OrderAmountHike),
    c.DaySinceLastOrder = COALESCE(c.DaySinceLastOrder, m.Mean_DaySinceLastOrder);
```

- Impute mode for the following columns: Tenure, CouponUsed, OrderCount.

```sql
WITH mode_values AS (
    SELECT
        (SELECT Tenure FROM customer_churn GROUP BY Tenure ORDER BY COUNT(*) DESC LIMIT 1) AS Mode_Tenure,
        (SELECT CouponUsed FROM customer_churn GROUP BY CouponUsed ORDER BY COUNT(*) DESC LIMIT 1) AS Mode_CouponUsed,
        (SELECT OrderCount FROM customer_churn GROUP BY OrderCount ORDER BY COUNT(*) DESC LIMIT 1) AS Mode_OrderCount
)
UPDATE customer_churn c
JOIN mode_values m
ON 1=1
SET c.Tenure = CASE WHEN c.Tenure IS NULL THEN m.Mode_Tenure ELSE c.Tenure END,
    c.CouponUsed = CASE WHEN c.CouponUsed IS NULL THEN m.Mode_CouponUsed ELSE c.CouponUsed END,
    c.OrderCount = CASE WHEN c.OrderCount IS NULL THEN m.Mode_OrderCount ELSE c.OrderCount END;
-- View changed values
SELECT Tenure, CouponUsed, OrderCount FROM customer_churn;
```

Tenure	CouponUsed	OrderCount
4	1	1
1	0	1
1	0	1
0	0	1
0	1	1
0	4	6
1	0	1
1	2	2

- Handle outliers in the 'WarehouseToHome' column by deleting rows where the values are greater than 100.

```sql
DELETE FROM customer_churn
WHERE WarehouseToHome > 100;
```

```sql
SELECT CustomerID,WarehouseToHome FROM customer_churn
WHERE WarehouseToHome > 100;
```

Dealing with Inconsistencies:

- Replace occurrences of “Phone” in the 'PreferredLoginDevice' column and “Mobile” in the ‘PreferedOrderCat' column with “Mobile Phone” to ensure uniformity.

```sql
SELECT PreferredLoginDevice,PreferedOrderCat FROM customer_churn;
```

PreferredLoginDevice	PreferedOrderCat
Mobile Phone	Laptop & Accessory
Phone	Mobile
Phone	Mobile
Phone	Laptop & Accessory
Phone	Mobile
Computer	Mobile Phone
Phone	Laptop & Accessory
Phone	Mobile

```sql
UPDATE customer_churn SET PreferredLoginDevice = 'Mobile Phone' WHERE PreferredLoginDevice = 'phone';
UPDATE customer_churn SET PreferedOrderCat = 'Mobile Phone' WHERE PreferedOrderCat = 'Mobile';
```

PreferredLoginDevice	PreferedOrderCat
Mobile Phone	Laptop & Accessory
Mobile Phone	Mobile Phone
Mobile Phone	Mobile Phone
Mobile Phone	Laptop & Accessory
Mobile Phone	Mobile Phone
Computer	Mobile Phone
Mobile Phone	Laptop & Accessory
Mobile Phone	Mobile Phone

- Standardize payment mode values: Replace "COD" with "Cash on Delivery" and "CC" with "Credit Card" in the PreferredPaymentMode column.

```sql
SELECT PreferredPaymentMode FROM customer_churn;
```

PreferredPaymentMode
Debit Card
COD
CC
Credit Card
UPI
Debit Card
E wallet
Debit Card

```sql
UPDATE customer_churn SET PreferredPaymentMode = 'COD' WHERE PreferredPaymentMode = 'Cash on Delivery';
UPDATE customer_churn SET PreferredPaymentMode = 'Credit Card' WHERE PreferredPaymentMode = 'CC';
```

PreferredPaymentMode
Debit Card
COD
Credit Card
Credit Card
UPI
Debit Card
E wallet
Debit Card

Data Transformation:

Column Renaming:

```sql
DESCRIBE customer_churn;
```

Field	Type	Null	Key	Default
HourSpendOnApp	int	YES		NULL
NumberOfDeviceRegistered	int	YES		NULL
PreferedOrderCat	varchar(20)	YES		NULL
SatisfactionScore	int	YES		NULL
MaritalStatus	varchar(10)	YES		NULL

- Rename the column "PreferedOrderCat" to "PreferredOrderCat".

```sql
ALTER TABLE customer_churn
RENAME COLUMN  PreferedOrderCat TO PreferredOrderCat;
```

- Rename the column "HourSpendOnApp" to "HoursSpentOnApp".

```sql
ALTER TABLE customer_churn
RENAME COLUMN  HourSpendOnApp TO HoursSpendOnApp;
```

Field	Type	Null	Key	Default
HoursSpendOnApp	int	YES		NULL
NumberOfDeviceRegistered	int	YES		NULL
PreferredOrderCat	varchar(20)	YES		NULL
SatisfactionScore	int	YES		NULL
MaritalStatus	varchar(10)	YES		NULL

Creating New Columns:

- Create a new column named 'ComplaintReceived' with values "Yes" if the corresponding value in the 'Complain' is 1, and "No" otherwise.

```
ALTER TABLE customer_churn ADD COLUMN ComplaintReceived VARCHAR(3);
```

Field	Type	Null	Key	Default
NumberOfAddress	int	YES		NULL
Complain	bit(1)	YES		NULL
OrderAmountHikeFromlast...	int	YES		NULL
CouponUsed	int	YES		NULL
OrderCount	int	YES		NULL
DaySinceLastOrder	int	YES		NULL
CashbackAmount	int	YES		NULL
ComplaintReceived	varchar(3)	YES		NULL

```
UPDATE customer_churn SET ComplaintReceived = CASE WHEN Complain = 1 THEN 'Yes' ELSE 'No' END;
```

```
SELECT Complain,ComplaintReceived FROM customer_churn;
```

Complain	ComplaintReceived
1	Yes
1	Yes
1	Yes
0	No
0	No
1	Yes
0	No

- Create a new column named 'ChurnStatus'. Set its value to "Churned" if the corresponding value in the 'Churn' column is 1, else assign "Active".

```sql
ALTER TABLE customer_churn ADD COLUMN ChurnStatus VARCHAR(10);
```

Field	Type	Null	Key	Default
Complain	bit(1)	YES		NULL
OrderAmountHikeFromlast...	int	YES		NULL
CouponUsed	int	YES		NULL
OrderCount	int	YES		NULL
DaySinceLastOrder	int	YES		NULL
CashbackAmount	int	YES		NULL
ComplaintReceived	varchar(3)	YES		NULL
ChurnStatus	varchar(10)	YES		NULL

```sql
UPDATE customer_churn
SET ChurnStatus =
    CASE
        WHEN Churn = 1 THEN 'Churned'
        ELSE 'Active'
    END;

SELECT Churn,ChurnStatus FROM customer_churn;
```

Churn	ChurnStatus
0	Active
0	Active
1	Churned
1	Churned
0	Active
0	Active
0	Active

Column Dropping:

- Drop the columns "Churn" and "Complain" from the table.

```
SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'ecomm' AND TABLE_NAME = 'customer_churn'
AND (COLUMN_NAME = 'Churn' OR COLUMN_NAME = 'Complain');
```

COLUMN_NAME
Churn
Complain

```
ALTER TABLE customer_churn
DROP COLUMN Churn;

ALTER TABLE customer_churn
DROP COLUMN Complain;
```

	COLUMN_NAME

Data Exploration and Analysis:

Retrieve the count of churned and active customers from the dataset.

```
SELECT COUNT(ChurnStatus) AS TotalChurn FROM customer_churn WHERE ChurnStatus= 'Churned' ;
SELECT COUNT(ChurnStatus) AS TotalActive FROM customer_churn WHERE ChurnStatus= 'Active' ;
```

TotalChurn
948

TotalActive
4680

```
SELECT ChurnStatus, COUNT(*) AS StatusCount FROM customer_churn
GROUP BY ChurnStatus;
```

ChurnStatus	StatusCount
Churned	948
Active	4680

Display the average tenure and total cashback amount of customers who churned.

```
SELECT AVG(Tenure) AS Avg_Churned_Tenure,SUM(CashbackAmount) AS Tot_Churned_CashbackAmount
FROM customer_churn WHERE ChurnStatus= 'Churned';
```

Avg_Churned_Tenure	Tot_Churned_CashbackAmount
3.1762	152030

Determine the percentage of churned customers who complained.

```sql
SELECT
    (
        (SELECT COUNT(CustomerID) FROM customer_churn WHERE ComplaintReceived = 'Yes' AND ChurnStatus = 'Churned') * 100.0
          /
        (SELECT COUNT(CustomerID) FROM customer_churn WHERE ChurnStatus = 'Churned' )
    ) AS ComplaintPercentage;
```

ComplaintPercentage
53.58650

```sql
SELECT (COUNT(CASE WHEN ComplaintReceived = 'Yes' THEN 1 END) * 100.0
        / COUNT(*)) AS Complaint_Percent_Churned
FROM customer_churn WHERE ChurnStatus = 'Churned';
```

Complaint_Percent_Churned
53.58650

Identify the city tier with the highest number of churned customers whose preferred order category is Laptop & Accessory.

```sql
SELECT CityTier, COUNT(*) AS No_Of_Cust FROM  customer_churn
WHERE ChurnStatus = 'Churned' AND PreferredOrderCat='Laptop & Accessory'
GROUP BY CityTier ORDER BY No_Of_Cust DESC LIMIT 1;
```

CityTier	No_Of_Cust
3	150

Identify the most preferred payment mode among active customers.

```sql
SELECT PreferredPaymentMode,count(*) AS No_Of_Pay_Mode FROM customer_churn
WHERE ChurnStatus= 'Active' GROUP BY  PreferredPaymentMode ORDER BY No_Of_Pay_Mode DESC
LIMIT 1 ;
```

PreferredPaymentMode	No_Of_Pay_Mode
Debit Card	1956

Calculate the total order amount hike from last year for customers who are single and prefer mobile phones for ordering.

```sql
SELECT MaritalStatus,PreferredLoginDevice,SUM(OrderAmountHikeFromlastYear) AS Tot_Order_Amt_Hike_LastYear FROM customer_churn
WHERE MaritalStatus = 'Single' AND PreferredLoginDevice = 'Mobile Phone'
GROUP BY MaritalStatus,PreferredLoginDevice;
```

MaritalStatus	PreferredLoginDevice	Tot_Order_Amt_Hike_LastYear
Single	Mobile Phone	19593

Find the average number of devices registered among customers who used UPI as their preferred payment mode.

```sql
SELECT AVG(NumberOfDeviceRegistered) AS Avg_No_Of_Devices_Registered FROM customer_churn
WHERE PreferredPaymentMode = 'UPI';
```

Avg_No_Of_Devices_Registered
3.7174

Determine the city tier with the highest number of customers.

```sql
SELECT CityTier,COUNT(*) AS No_Of_Customer FROM customer_churn
GROUP BY CityTier ORDER BY No_Of_Customer DESC LIMIT 1;
```

CityTier	No_Of_Customer
1	3666

Identify the gender that utilized the highest number of coupons.

```sql
select Gender,SUM(CouponUsed) AS Tot_CouponUsed from customer_churn
GROUP BY Gender ORDER BY Tot_CouponUsed DESC LIMIT 1 ;
```

Gender	Tot_CouponUsed
Male	5629

List the number of customers and the maximum hours spent on the app in each preferred order category.

```sql
select PreferredOrderCat,COUNT(*) AS  No_of_Cust,MAX(HoursSpendOnApp) AS Max_Hours_Spend_On_App from customer_churn
GROUP BY PreferredOrderCat ORDER BY Max_Hours_Spend_On_App DESC ;
```

PreferredOrderCat	No_of_Cust	Max_Hours_Spend_On_App
Laptop & Accessory	2050	5
Mobile Phone	2078	5
Fashion	826	5
Others	264	4
Grocery	410	4

Calculate the total order count for customers who prefer using credit cards and have the maximum satisfaction score.

```
select SUM(OrderCount) AS Tot_Order_Cust from customer_churn
WHERE PreferredPaymentMode = 'Credit Card' AND
SatisfactionScore = (
      SELECT MAX(SatisfactionScore)
      FROM customer_churn
      WHERE PreferredPaymentMode = 'Credit Card'
);
```

Tot_Order_Cust
1122

What is the average satisfaction score of customers who have complained?

```
select AVG(SatisfactionScore) AS Avg_Satisfication_Score from customer_churn
WHERE ComplaintReceived= 'Yes';
```

Avg_Satisfication_Score
2.9988

List the preferred order category among customers who used more than 5 coupons.

```
SELECT PreferredOrderCat,CouponUsed from customer_churn
WHERE CouponUsed > 5 ORDER BY CouponUsed ;
```

PreferredOrderCat	CouponUsed
Laptop & Accessory	6
Laptop & Accessory	6
Laptop & Accessory	6
Laptop & Accessory	6
Fashion	6
Laptop & Accessory	6
Mobile Phone	6

```
SELECT DISTINCT PreferredOrderCat from customer_churn
WHERE CouponUsed > 5;
```

PreferredOrderCat
Others
Laptop & Accessory
Fashion
Mobile Phone
Grocery

List the top 3 preferred order categories with the highest average cashback amount.

```
SELECT PreferredOrderCat,AVG(CashbackAmount) AS Avg_Cashback_Amt  from customer_churn
GROUP BY PreferredOrderCat ORDER BY Avg_Cashback_Amt DESC LIMIT 3;
```

PreferredOrderCat	Avg_Cashback_Amt
Others	304.4545
Grocery	266.2366
Fashion	210.4031

Find the preferred payment modes of customers whose average tenure is 10 months and have placed more than 500 orders.

```
SELECT PreferredPaymentMode,AVG(Tenure) AS Avg_Tenure,SUM(OrderCount) AS Tot_Orders from customer_churn
GROUP BY PreferredPaymentMode HAVING Avg_Tenure > 10 AND Tot_Orders > 500;
```

PreferredPaymentMode	Avg_Tenure	Tot_Orders
E wallet	10.1954	1849

Categorize customers based on their distance from the warehouse to home such as 'Very Close Distance' for distances <=5km, 'Close Distance' for <=10km, 'Moderate Distance' for <=15km, and 'Far Distance' for >15km. Then, display the churn status breakdown for each distance category.

```sql
SELECT
    CASE
        WHEN WarehouseToHome <= 5 THEN 'Very Close Distance'
        WHEN WarehouseToHome <= 10 THEN 'Close Distance'
        WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'
        ELSE 'Far Distance'
    END AS DistanceCategory,
ChurnStatus, COUNT(*) AS customer_count FROM customer_churn
GROUP BY DistanceCategory, ChurnStatus
ORDER BY DistanceCategory, ChurnStatus;
```

DistanceCategory	ChurnStatus	customer_count
Close Distance	Active	1696
Close Distance	Churned	265
Far Distance	Active	1871
Far Distance	Churned	498
Moderate Distance	Active	1106
Moderate Distance	Churned	184
Very Close Distance	Active	7
Very Close Distance	Churned	1

List the customer's order details who are married, live in City Tier-1, and their order counts are more than the average number of orders placed by all customers.

a) Create a 'customer_returns' table in the 'ecomm' database and insert the following data

```
CREATE TABLE customer_returns AS
SELECT * FROM customer_churn WHERE 1=0;
SHOW TABLES;
```

Tables_in_ecomm
customer_churn
customer_returns

```
INSERT INTO customer_returns
SELECT * from customer_churn
WHERE MaritalStatus = 'Married' AND CityTier='1' AND
OrderCount >  (
    SELECT AVG(OrderCount) FROM customer_churn
);
SELECT * FROM customer_returns;
```

CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	Hours
50049	3	Computer	1	15	Credit Card	Male	3
50119	13	Mobile Phone	1	30	COD	Male	3
50132	15	Mobile Phone	1	16	Credit Card	Female	2
50133	13	Mobile Phone	1	8	Credit Card	Male	2
50152	2	Mobile Phone	1	28	Debit Card	Female	3
50180	12	Mobile Phone	1	6	Credit Card	Male	2
50187	18	Computer	1	13	Debit Card	Male	3

b) Display the return details along with the customer details of those who have churned and have made complaints.

```
SELECT * FROM customer_returns
WHERE ChurnStatus = 'Churned' AND ComplaintReceived='Yes';
```

CashbackAmount	ComplaintReceived	ChurnStatus
120	Yes	Churned
133	Yes	Churned
123	Yes	Churned
150	Yes	Churned